# Memoria Proyecto Final de Grado: PremadeGG

Matti Johannes Ahola Rivas

22 de febrero de 2026

## Índice

**1. Introducción y Propuesta de Valor** **2**

**2. Stack Tecnológico (MEAN) y Herramientas** **2**

**3. Arquitectura Global del Proyecto** **3**
3.1. Arquitectura del Backend (`/server`) . . . 3
3.2. Arquitectura del Frontend (`/client`) . . . 3

**4. Modelos de Datos Exhaustivos** **3**
4.1. Usuario (User) . . . 4
4.2. Equipo (Team) . . . 4
4.3. Tablón de Anuncios (LFGPost) . . . 4
4.4. Reseñas y Karma (Reputation) . . . 5
4.5. Comunicación (Conversation, Message, Notification) . . . 5

**5. Flujos de Funcionamiento y Casos de Uso** **5**
5.1. Registro, Autenticación y Recuperación . . . 5
5.2. Sistema de Reputación y Exploración de Jugadores . . . 5
5.3. Ecosistema de Equipos y Tablón LFG . . . 6
5.4. Tiempo Real: Chat y Notificaciones . . . 6

**6. Integración Externa y Garantía de Datos** **6**

**7. Arquitectura Front-End Angular** **6**

**8. Seguridad Integral** **7**

**9. Despliegue, Infraestructura y DevOps** **7**

**10. Conclusión: Resumen del Ciclo de Vida del Jugador** **8**

# 1. Introducción y Propuesta de Valor

PremadeGG es una plataforma "Looking For Group" (LFG) diseñada específicamente para la comunidad de jugadores de Valorant. El ecosistema actual de los juegos competitivos a menudo adolece de problemas de comportamiento y toxicidad que arruinan la experiencia de usuario. El objetivo fundamental de esta plataforma es resolver este problema al conectar jugadores basándose no solo en su rango competitivo, sino también en su reputación (medida a través de un sistema de "Karma") y en la compatibilidad de sus roles dentro del juego.

La principal innovación del proyecto es su **Sistema Anti-Toxicidad** proactivo. La plataforma establece una regla de filtrado estricta: los jugadores que posean un Karma inferior a 3 son automáticamente filtrados del matchmaking principal. Esta mecánica desincentiva el mal comportamiento y fomenta la creación de una comunidad sana, colaborativa y libre de toxicidad.

# 2. Stack Tecnológico (MEAN) y Herramientas

El desarrollo de PremadeGG se apoya en el robusto stack MEAN, seleccionado por su alto rendimiento, uso unificado de JavaScript/TypeScript y su capacidad para manejar operaciones asíncronas en tiempo real. Se complementa con servicios externos especializados:

- **Base de Datos (MongoDB + Mongoose):** Seleccionada por su esquema flexible (NoSQL), ideal para almacenar documentos complejos como usuarios con arrays variables, equipos, posts LFG, mensajes, notificaciones y reseñas cruzadas.
- **Backend (Express.js + Node.js):** Proporciona una API RESTful estructurada mediante una arquitectura estricta de Modelo-Vista-Controlador (MVC).
- **Frontend (Angular):** Desarrollado utilizando la última arquitectura de *Standalone Components* (sin NgModules), lo que permite crear una Single Page Application (SPA) altamente reactiva. El diseño visual se implementa con SASS (metodología 7-1), aplicando una estética de modo oscuro y *glassmorphism*.
- **Comunicación en Tiempo Real (Socket.IO):** Fundamental para establecer conexiones bidireccionales, permitiendo la entrega instantánea de notificaciones y el funcionamiento fluido de la mensajería in-app.
- **Servicio de Emailing (Brevo/Sendinblue):** Sustituye a soluciones SMTP tradicionales (que suelen estar bloqueadas en capas gratuitas de hosting) para gestionar el envío transaccional de correos, específicamente para la recuperación de contraseñas.
- **Integración de API Externa (Riot Games API vía Henrik Dev):** Componente vital para la validación verídica de identidades de cuentas y la obtención precisa del rango competitivo real de los jugadores.

# 3. Arquitectura Global del Proyecto

El proyecto está estructurado bajo un patrón de monorepo, dividiendo claramente las responsabilidades entre el servidor y el cliente:

## 3.1. Arquitectura del Backend (`/server`)

El servidor funciona como el punto de entrada a través de `app.js`, e implementa una arquitectura MVC rigurosa:

- **Conexión de Datos:** La configuración de acceso a MongoDB se centraliza en `config/db.js`.
- **Modelos y Controladores:** Dispone de 7 modelos Mongoose que definen los esquemas de la base de datos y 9 controladores que encapsulan la lógica de negocio pura.
- **Rutas:** 9 ficheros dedicados gestionan la exposición de los endpoints de la API.
- **Middlewares y Servicios:** Incluye middlewares de autenticación basados en JSON Web Tokens (JWT) y manejo centralizado de errores. La lógica de terceros se extrae a servicios dedicados (Email, Riot API, Socket.IO).
- **Utilidades:** Cuenta con scripts de inicialización (`seedDefaults.js`) para inyectar datos de prueba seguros durante el desarrollo.

## 3.2. Arquitectura del Frontend (`/client`)

El cliente Angular se organiza para maximizar la reutilización y el rendimiento mediante *lazy loading*:

- **Páginas (Pages):** 13 vistas independientes que se cargan bajo demanda.
- **Componentes Compartidos (Shared):** 8 componentes UI reutilizables (tarjetas, medallas, barras de navegación) que mantienen la consistencia visual.
- **Servicios (Services):** 5 servicios inyectables para centralizar llamadas HTTP, WebSockets y estados de autenticación.
- **Seguridad y Modelos:** Protegido mediante un *Guard* de autenticación en las rutas y un *Interceptor* HTTP para inyectar automáticamente los JWT. El tipado se garantiza mediante 8 interfaces TypeScript.

# 4. Modelos de Datos Exhaustivos

La estructura de la base de datos es el núcleo de las interacciones sociales de la plataforma.

## 4.1. Usuario (User)

Almacena el perfil integral del jugador de forma segura:

- **Credenciales y Autenticación:** Correo electrónico, contraseña (fuertemente hasheada con `bcrypt`) y tokens de recuperación temporal.
- **Identidad y Datos de Juego:** Almacena el Riot ID oficial (ej. Player#1234), el PUUID interno, la región de juego (eu, na, ap, kr). Registra el rango competitivo (desde Hierro hasta Radiante), el rol principal (Duelista, Controlador, Iniciador, Centinela, Flex), su horario habitual y un máximo de 3 agentes favoritos.
- **Sistema de Karma:** Valor numérico del 0 al 5 (iniciando en 5 por defecto), que se actualiza dinámicamente calculando la media matemática de las reseñas recibidas.
- **Personalización y Búsqueda:** Biografía, imagen de avatar, enlaces a redes sociales (Discord, Twitter, Twitch), mapa de disponibilidad diaria e indicadores booleanos como `lookingForTeam` y `isCoach`.

## 4.2. Equipo (Team)

Gestiona la formación de clanes o escuadras competitivas:

- **Identidad y Configuración:** Nombre único (máximo 40 caracteres), etiqueta o *tag* corto (máximo 6 caracteres), logotipo, región de actuación, rango competitivo objetivo, enlaces externos y listado de logros.
- **Estructura Jerárquica:** Diferencia claramente entre el propietario (`owner`), el entrenador (`coach`), la plantilla de jugadores (con roles asignados dentro del equipo) y el personal auxiliar o *staff* (managers, analistas, creadores de contenido, streamers).
- **Sistema de Reclutamiento:** Mediante un flag booleano de reclutamiento, detalla qué roles se buscan y maneja un array interno de `joinRequests` con los estados transicionales (pendiente, aceptado, rechazado).

## 4.3. Tablón de Anuncios (LFGPost)

Representa las solicitudes efímeras para jugar en grupo:

- **Datos de la Publicación:** Referencia al autor, un título descriptivo, cuerpo del texto, tamaño deseado de la escuadra (2, 3 o 5 jugadores), y filtros estrictos de rango y karma mínimos (por defecto 3).
- **Gestión de Solicitudes:** Un array rastrea a los usuarios interesados y su estado de aprobación (pendiente/aceptado). El estado global del post alterna entre abierto, lleno o cerrado.
- **Expiración Automática (TTL):** Implementa un índice de *Time-To-Live* (TTL) en MongoDB que asegura que los posts caduquen y sean eliminados automáticamente a las 24 horas de su creación.

### 4.4. Reseñas y Karma (Reputation)

El motor detrás del sistema anti-toxicidad:

- **Relación Dirigida:** Almacena quién escribe la reseña (`author`) y quién la recibe (`target`).
- **Evaluación:** Incluye una puntuación del 1 al 5, un comentario escrito y un validador de que realmente han jugado juntos (`playedTogether`).
- **Prevención de Abuso:** Se aplica un índice compuesto único a nivel de base de datos que garantiza que un autor solo pueda evaluar a un mismo objetivo una única vez.

### 4.5. Comunicación (Conversation, Message, Notification)

- **Mensajería:** Soporte robusto tanto para chats directos 1-a-1 como para conversaciones grupales. Para optimizar las consultas, cada conversación guarda un registro de sus participantes, una instantánea del último mensaje enviado y contadores de lectura para calcular los mensajes no leídos.
- **Notificaciones:** Alertas de aplicación (solicitudes de unión, nuevas reseñas, mensajes) que cuentan con estados de lectura y se empujan en tiempo real hacia el cliente.

## 5. Flujos de Funcionamiento y Casos de Uso

### 5.1. Registro, Autenticación y Recuperación

El ciclo de vida comienza con un proceso de registro seguro donde el usuario proporciona credenciales y su Riot ID. Antes de crear la cuenta, el backend intercepta la petición y valida este Riot ID contra la API oficial. Tras verificar su existencia, extrae automáticamente el rango real y finaliza el registro, devolviendo un JWT. Los inicios de sesión subsecuentes verifican las credenciales vía bcrypt y despachan un nuevo JWT. Todo el sistema opera sin estado (*stateless*); el JWT se envía en la cabecera `Authorization: Bearer <token>` de cada petición protegida.

En caso de olvido, el usuario solicita un restablecimiento que genera un token seguro de un solo uso. La API de Brevo interviene para enviar un correo electrónico transaccional con el enlace de recuperación, que permite al usuario actualizar su contraseña mediante una nueva petición verificada.

### 5.2. Sistema de Reputación y Exploración de Jugadores

La vista de exploración de jugadores presenta una lista continua (optimizada mediante *scroll infinito* y paginación por cursor). Permite búsquedas directas por Riot ID y presenta a los usuarios en tarjetas ("Player Cards") que resumen su rango, rol, agentes y nivel de karma. Para garantizar un entorno seguro, el frontend filtra por defecto a todo jugador cuyo Karma sea inferior a 3.

El flujo de reputación exige que se verifique la elegibilidad antes de permitir valorar a alguien. Tras enviar la evaluación (1-5 estrellas), el servidor recalcula de inmediato el

karma global del objetivo. Si esta nueva media desciende de 3, el jugador pierde visibilidad en el ecosistema. Las restricciones de base de datos evitan el "review bombing" de una misma persona.

### 5.3. Ecosistema de Equipos y Tablón LFG

El **Tablón LFG** está pensado para sesiones inmediatas. Un jugador crea una oferta detallando el tamaño del grupo y los requisitos mínimos. El post se publica en estado abierto y permite a otros solicitar unirse. El creador actúa como administrador, aprobando o rechazando candidatos. Si se alcanza la cuota, pasa a estado lleno, y transcurridas 24 horas, desaparece por completo sin requerir mantenimiento.

Para relaciones a largo plazo, los **Equipos** ofrecen una estructura persistente. El creador adquiere el rol de propietario y define la identidad del clan. Las solicitudes de entrada entran en una cola pendiente hasta ser auditadas y resueltas por el dueño. Los miembros aceptados pasan a la plantilla oficial, y la gestión avanzada permite nombrar entrenadores o personal administrativo.

### 5.4. Tiempo Real: Chat y Notificaciones

La plataforma elimina la fricción de requerir aplicaciones externas de terceros. Al iniciar un chat directo y enviar un texto, el backend lo procesa y lo intercepta mediante Socket.IO. Si el destinatario está online, recibe la entrega de manera instantánea; simultáneamente, los contadores de mensajes no leídos se incrementan automáticamente en la base de datos. Este comportamiento es idéntico para los salones de grupo.

Las interacciones de alto valor (ser aceptado en un equipo, recibir una valoración) generan notificaciones persistentes que activan una alerta visual en la campana de navegación, también transmitidas en tiempo real.

## 6. Integración Externa y Garantía de Datos

La calidad de los datos competitivos se garantiza mediante la interconexión con Henrik Dev, una envoltura sobre la Riot Games API. Se utilizan dos endpoints principales:

1. **Validación de cuenta** (`GET /api/riot/validate/:gameName/:tagLine`), que asegura que nadie pueda registrarse usando identidades falsas o nombres inventados.

2. **Obtención de MMR/Rango** (`GET /api/riot/rank/:region/:gameName/:tagLine`), que asegura que las medallas de rango exhibidas en las tarjetas de jugador son verídicas y corresponden al nivel competitivo actual del usuario.

## 7. Arquitectura Front-End Angular

La interfaz visual se divide funcionalmente en una colección de páginas enrutadas de forma perezosa y componentes agnósticos:

- **Rutas Clave:** Desde la *Home* (`/`) promocional, hasta listados con filtros complejos (`/players`, `/teams`), el tablón temporal (`/lfg`) y las vistas privadas como la bandeja de entrada (`/messages`) o la configuración de perfil (`/settings/profile`). Las rutas privadas bloquean el acceso a usuarios no identificados mediante Guardias Angular.
- **Componentes Reutilizables:** Piezas atómicas como `Navbar` (con integración de perfil), `PlayerCard` y `TeamCard` (para estandarizar la visualización de perfiles y clanes), y micro-componentes de estado como `KarmaBadge` (cambia de color dinámicamente según la nota), `RankBadge` y `NotificationBell`.
- **Servicios Lógicos:** `AuthService` gestiona el almacenamiento en memoria del JWT y la persistencia de sesión. `ApiService` actúa como adaptador HTTP central con una capa de caché en memoria de 60 segundos para evitar recargas excesivas. `SocketService` y `MessageService` / `NotificationService` manejan toda la complejidad de WebSockets y *polling* en tiempo real.

# 8. Seguridad Integral

El proyecto incorpora múltiples capas de protección defensiva:

- Autenticación descentralizada mediante tokens JWT firmados criptográficamente.
- Hasheo irreversible de contraseñas mediante `bcrypt` antes de cualquier transacción en la base de datos.
- Política estricta de CORS configurada para rechazar cualquier petición de producción que no provenga del dominio oficial de la aplicación alojada en Vercel.
- Esquemas de validación de Mongoose profundamente tipados, aplicando reglas de longitud, enumeraciones cerradas y restricciones de unicidad.
- Middlewares globales de Express.js diseñados para capturar errores, evitar exposición de la traza del stack en producción y registrar actividad anómala mediante Morgan.

# 9. Despliegue, Infraestructura y DevOps

El ciclo de vida del desarrollo está completamente contenedorizado para el entorno local y automatizado para la nube:

- **Local (Docker):** A través de `docker-compose`, se levanta en conjunto la API backend (puerto 4000), una instancia de MongoDB (27017) y la herramienta visual Mongo Express (8082). Un script dedicado (`npm run seed`) permite inyectar conjuntos masivos de datos controlados para probar paginación y filtros inmediatamente.

- **Producción en la Nube:** Todo el despliegue es altamente escalable. El código cliente pre-compilado de Angular se distribuye a través de la red perimetral de Vercel, mientras que el clúster de base de datos reside en MongoDB Atlas. Finalmente, la lógica en Node.js se ejecuta sobre el servicio PaaS de Render, conectado mediante variables de entorno seguras.

# 10. Conclusión: Resumen del Ciclo de Vida del Jugador

El impacto del sistema se resume en el ciclo iterativo que experimenta un jugador en PremadeGG. Comienza superando el escrutinio del registro verificando su Riot ID y rango real. Pasa a explorar el mercado de jugadores aplicando filtros avanzados o crea publicaciones efímeras para encontrar grupo de manera inmediata. Para un entorno estructurado, interactúa con el ecosistema de equipos competitivos.

Tras las partidas, interviene en la evaluación de la comunidad intercambiando reseñas. Utiliza la mensajería interna y recibe alertas sin fricciones para coordinar futuros encuentros. Finalmente, si el jugador exhibe conductas nocivas, el sistema interviene: al degradar su Karma por debajo del umbral de 3, lo aísla algorítmicamente del matchmaking principal para preservar la integridad de todo el ecosistema.